



데이터 불러오기

```
In [1]: !pip install torchinfo
!pip install torchmetrics matplotlib
import numpy as np
import matplotlib.pyplot as plt
from torchvision import datasets, transforms
from torchinfo import summary

# 1. 데이터 불러오기
# For displaying, we might want the raw images, so loading again without
normalize for visualization
raw_train_dataset = datasets.MNIST(root='./data', train=True,
download=True, transform=transforms.ToTensor())
raw_test_dataset = datasets.MNIST(root='./data', train=False,
download=True, transform=transforms.ToTensor())

# Extract data for shape printing and visualization
x_train_raw = raw_train_dataset.data.numpy()
y_train_raw = raw_train_dataset.targets.numpy()
x_test_raw = raw_test_dataset.data.numpy()
y_test_raw = raw_test_dataset.targets.numpy()

print("x_train shape:", x_train_raw.shape)
print("y_train shape:", y_train_raw.shape)
print("x_test shape:", x_test_raw.shape)
print("y_test shape:", y_test_raw.shape)

# 2. 임의의 100개 데이터 인덱스 선택
indices = np.random.choice(len(x_train_raw), size=100, replace=False)

# 3. 10x10 격자로 이미지 출력
fig, axes = plt.subplots(10, 10, figsize=(10, 10))
for i, index in enumerate(indices):
    row = i // 10
    col = i % 10
    # PyTorch datasets return PIL images or tensors, need to convert to
numpy for imshow
    # If using raw_train_dataset.data, it's already a numpy array
    axes[row, col].imshow(x_train_raw[index], cmap='gray')
    axes[row, col].axis('off')
plt.suptitle("MNIST Training Data (100 samples) - PyTorch", fontsize=16)
plt.tight_layout()
plt.show()
```

Collecting torchinfo

Downloading torchinfo-1.8.0-py3-none-any.whl.metadata (21 kB)

Downloading torchinfo-1.8.0-py3-none-any.whl (23 kB)

Installing collected packages: torchinfo

Successfully installed torchinfo-1.8.0

Collecting torchmetrics

Downloading torchmetrics-1.9.0-py3-none-any.whl.metadata (23 kB)

Requirement already satisfied: matplotlib in

```
/usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: numpy>1.20.0 in
/usr/local/lib/python3.12/dist-packages (from torchmetrics)
(2.0.2)
Requirement already satisfied: packaging>17.1 in
/usr/local/lib/python3.12/dist-packages (from torchmetrics) (26.2)
Requirement already satisfied: torch>=2.0.0 in
/usr/local/lib/python3.12/dist-packages (from torchmetrics)
(2.11.0+cu128)
Collecting lightning-utilities>=0.15.3 (from torchmetrics)
  Downloading lightning_utilities-0.15.3-py3-none-any.whl.metadata
(5.5 kB)
Requirement already satisfied: contourpy>=1.0.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: pillow>=8 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in
/usr/local/lib/python3.12/dist-packages (from matplotlib)
(2.9.0.post0)
Requirement already satisfied: typing_extensions in
/usr/local/lib/python3.12/dist-packages (from lightning-
utilities>=0.15.3->torchmetrics) (4.15.0)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.12/dist-packages (from python-
dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: filelock in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0-
>torchmetrics) (3.29.0)
Requirement already satisfied: setuptools<82 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0-
>torchmetrics) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0-
>torchmetrics) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0-
>torchmetrics) (3.6.1)
Requirement already satisfied: jinja2 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0-
>torchmetrics) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in
/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0-
>torchmetrics) (2025.3.0)
Requirement already satisfied: cuda-toolkit==12.8.1 in
/usr/local/lib/python3.12/dist-packages (from cuda-
toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,
nvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"-
>torch>=2.0.0->torchmetrics) (12.8.1)
Requirement already satisfied: cuda-bindings<13,>=12.9.4 in
```

/usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->torchmetrics) (12.9.6)
Requirement already satisfied: nvidia-cudnn-cu12==9.19.0.56 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->torchmetrics) (9.19.0.56)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->torchmetrics) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.28.9 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->torchmetrics) (2.28.9)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->torchmetrics) (3.4.5)
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->torchmetrics) (3.6.0)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparselt,nvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch>=2.0.0->torchmetrics) (12.8.4.1)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparselt,nvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch>=2.0.0->torchmetrics) (12.8.90)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparselt,nvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch>=2.0.0->torchmetrics) (11.3.3.83)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparselt,nvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch>=2.0.0->torchmetrics) (1.13.1.3)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparselt,nvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch>=2.0.0->torchmetrics) (12.8.90)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparselt,nvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch>=2.0.0->torchmetrics) (10.3.9.90)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparselt,nvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch>=2.0.0->torchmetrics) (11.7.3.90)
Requirement already satisfied: nvidia-cusparselt-cu12==12.5.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparselt,nvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"->torch>=2.0.0->torchmetrics) (12.5.8.93)

```

Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93.* in
/usr/local/lib/python3.12/dist-packages (from cuda-
toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,
nvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"-
>torch>=2.0.0->torchmetrics) (12.8.93)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93.*
in /usr/local/lib/python3.12/dist-packages (from cuda-
toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,
nvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"-
>torch>=2.0.0->torchmetrics) (12.8.93)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90.* in
/usr/local/lib/python3.12/dist-packages (from cuda-
toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,
nvjitlink,nvrtc,nvtx]==12.8.1; platform_system == "Linux"-
>torch>=2.0.0->torchmetrics) (12.8.90)
Requirement already satisfied: cuda-pathfinder~=1.1 in
/usr/local/lib/python3.12/dist-packages (from cuda-
bindings<13,>=12.9.4->torch>=2.0.0->torchmetrics) (1.5.4)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3-
>torch>=2.0.0->torchmetrics) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.12/dist-packages (from jinja2-
>torch>=2.0.0->torchmetrics) (3.0.3)
Downloading torchmetrics-1.9.0-py3-none-any.whl (983 kB)
  2K 90m-----0m
32m983.4/983.4 kB0m 31m16.8 MB/s0m eta 36m0:00:000m
25hDownloading lightning_utilities-0.15.3-py3-none-any.whl (31
kB)
Installing collected packages: lightning-utilities, torchmetrics
Successfully installed lightning-utilities-0.15.3 torchmetrics-
1.9.0

```

```

100%|██████████| 9.91M/9.91M [00:00<00:00, 60.5MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 1.70MB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 14.8MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 7.94MB/s]

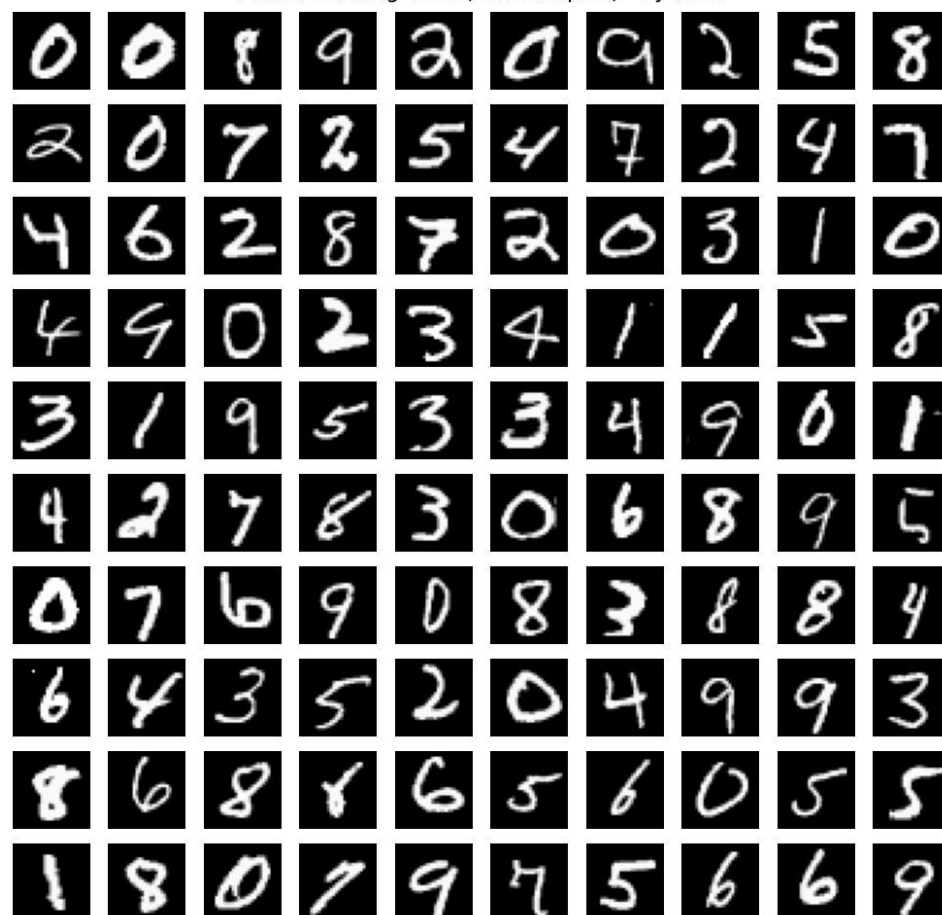
```

```

x_train shape: (60000, 28, 28)
y_train shape: (60000,)
x_test shape: (10000, 28, 28)
y_test shape: (10000,)

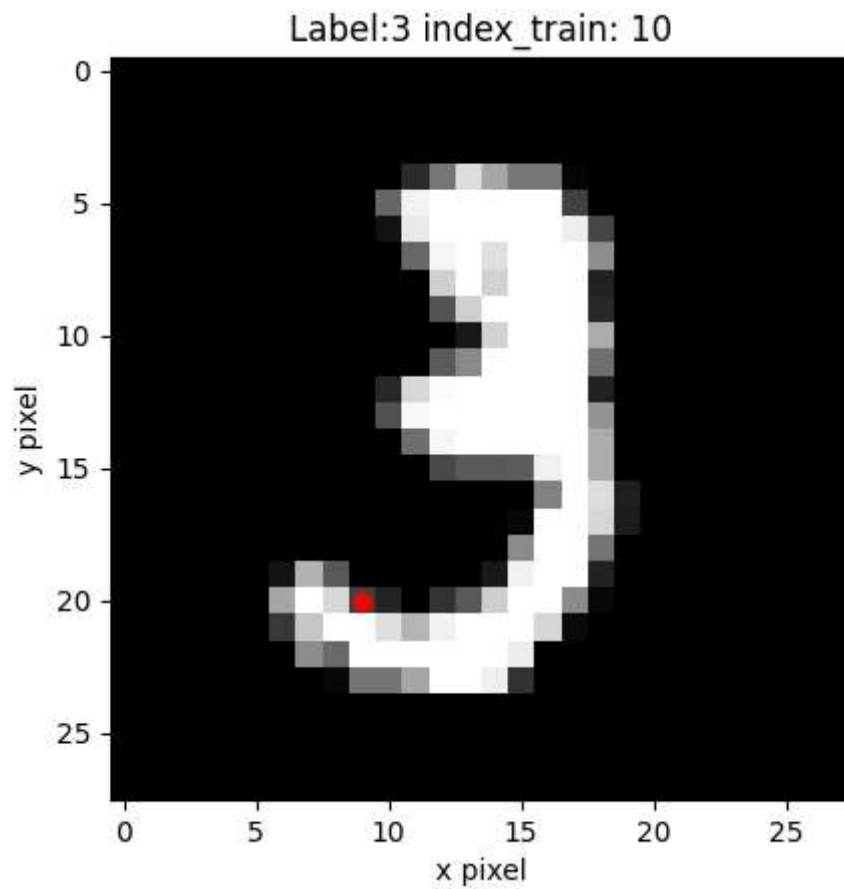
```

MNIST Training Data (100 samples) - PyTorch



```
In [2]: x_px1 = 9# @param {type:"raw"}
y_px1 = 20# @param {type:"raw"}
index_train = 10# @param {type:"raw"}
plt.imshow(x_train_raw[index_train], cmap='gray')
plt.xlabel("x pixel")
plt.ylabel("y pixel")
plt.plot(x_px1,y_px1,'ro')
plt.title(f"Label:{y_train_raw[index_train]} index_train:
{index_train}")
print(f'The value of (x={x_px1},y={y_px1}) is',
      x_train_raw[index_train][y_px1,x_px1])
```

The value of (x=9,y=20) is 63



신경망 만들기

```
In [3]: import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader, random_split
from tqdm import tqdm

# 4. MLP 모델 정의 (PyTorch)
class Perceptron(nn.Module):
    def __init__(self):
        super(Perceptron, self).__init__()
        self.fc1 = nn.Linear(28 * 28, 10) # Input: 784, Output: 128
    def forward(self, x):
        x = x.view(-1, 28*28)
        x = self.fc1(x)
        return x # Logits

class MLP_naive(nn.Module):
    def __init__(self):
        super(MLP_naive, self).__init__()
        self.fc1 = nn.Linear(28 * 28, 128) # Input: 784, Output: 128
        self.fc2 = nn.Linear(128, 64)      # Input: 128, Output: 64
        self.fc3 = nn.Linear(64, 10)       # Input: 64, Output: 10 (for
0-9 classes)

    def forward(self, x):
        x = x.view(-1, 28*28)
        x = self.fc1(x)
        x = self.fc2(x)
        x = self.fc3(x)
        return x # Logits

class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()
        self.fc1 = nn.Linear(28 * 28, 128) # Input: 784, Output: 128
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(128, 64)      # Input: 128, Output: 64
        self.relu2 = nn.ReLU()
        self.fc3 = nn.Linear(64, 10)       # Input: 64, Output: 10 (for
0-9 classes)

    def forward(self, x):
        x = x.view(-1, 28*28)
        x = self.fc1(x)
        x = self.relu1(x)
        x = self.fc2(x)
        x = self.relu2(x)
        x = self.fc3(x)
        return x # Logits

class FlexibleMLPSequential(nn.Module):
    def __init__(self, layer_sizes):
```



```

super(FlexibleMLPSequential, self).__init__()

# Build a list of layers first
modules = []
for i in range(len(layer_sizes) - 1):
    modules.append(nn.Linear(layer_sizes[i], layer_sizes[i+1]))
    if i < len(layer_sizes) - 2:
        modules.append(nn.ReLU())

# Unpack the list into nn.Sequential
# This registers everything at once and defines the forward pass
order
self.network = nn.Sequential(*modules)

def forward(self, x):
    # Flatten the input automatically based on the first layer's
input features
    # self.network[0] points to the very first nn.Linear layer
    x = x.view(-1, self.network[0].in_features)

    # No need for loops in forward; nn.Sequential handles it
internally
    return self.network(x)
class HighPerfCNN(nn.Module):
    def __init__(self):
        super(HighPerfCNN, self).__init__()

        # Input shape: (batch_size, 1, 28, 28)
        # First convolutional block
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=32,
kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(32)
        self.relu1 = nn.ReLU()

        # Second convolutional block
        self.conv2 = nn.Conv2d(in_channels=32, out_channels=64,
kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(64)
        self.relu2 = nn.ReLU()

        # Max pooling downsamples 28x28 to 14x14
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)
        self.dropout1 = nn.Dropout(0.25)

        # Third convolutional block
        self.conv3 = nn.Conv2d(in_channels=64, out_channels=128,
kernel_size=3, padding=1)
        self.bn3 = nn.BatchNorm2d(128)
        self.relu3 = nn.ReLU()

        # Max pooling downsamples 14x14 to 7x7
        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)
        self.dropout2 = nn.Dropout(0.25)

        # Fully connected layers
        # After two pools, the feature map size is 128 channels * 7 * 7
        self.fc1 = nn.Linear(128 * 7 * 7, 256)

```

```

        self.bn4 = nn.BatchNorm1d(256)
        self.relu4 = nn.ReLU()
        self.dropout3 = nn.Dropout(0.5)

        # Final output layer (10 classes for digits 0-9)
        self.fc2 = nn.Linear(256, 10)

    def forward(self, x):
        # Feature extraction
        x = self.relu1(self.bn1(self.conv1(x)))
        x = self.relu2(self.bn2(self.conv2(x)))
        x = self.dropout1(self.pool1(x))

        x = self.relu3(self.bn3(self.conv3(x)))
        x = self.dropout2(self.pool2(x))

        # Flatten the feature maps for the dense layers
        x = x.view(-1, 128 * 7 * 7)

        # Classification
        x = self.dropout3(self.relu4(self.bn4(self.fc1(x))))
        x = self.fc2(x)

        return x # Logits

import torch
import torch.nn as nn

class LowParamCNN(nn.Module):
    def __init__(self):
        super(LowParamCNN, self).__init__()

        # Input shape: (batch_size, 1, 28, 28)
        # First convolutional block: reduce channels to 16
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=16,
kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(16)
        self.relu1 = nn.ReLU()

        # Second convolutional block: reduce channels to 32
        self.conv2 = nn.Conv2d(in_channels=16, out_channels=32,
kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(32)
        self.relu2 = nn.ReLU()

        # Max pooling downsamples 28x28 to 14x14
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)
        self.dropout1 = nn.Dropout(0.2)

        # Third convolutional block: reduce channels to 64
        self.conv3 = nn.Conv2d(in_channels=32, out_channels=64,
kernel_size=3, padding=1)
        self.bn3 = nn.BatchNorm2d(64)
        self.relu3 = nn.ReLU()

        # Max pooling downsamples 14x14 to 7x7
        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)
        self.dropout2 = nn.Dropout(0.2)

```

```

        # Global Average Pooling: collapses (batch, 64, 7, 7) to (batch,
64, 1, 1)
        self.gap = nn.AdaptiveAvgPool2d((1, 1))

        # Final output layer: direct mapping from 64 channels to 10
classes
        # This drastically removes millions of parameters from the
previous FC layers
        self.fc = nn.Linear(64, 10)

    def forward(self, x):
        x = self.relu1(self.bn1(self.conv1(x)))
        x = self.relu2(self.bn2(self.conv2(x)))
        x = self.dropout1(self.pool1(x))

        x = self.relu3(self.bn3(self.conv3(x)))
        x = self.dropout2(self.pool2(x))

        # Apply GAP and flatten for the single linear layer
        x = self.gap(x)
        x = torch.flatten(x, 1)

        x = self.fc(x)
        return x

```

```

In [4]: import torchmetrics
model_dict = dict()
device_gpu = torch.device("cuda" if torch.cuda.is_available() else
"cpu")
device_cpu = torch.device("cpu")
device = device_gpu
train_size = 50000
val_size = 10000
generator = torch.Generator().manual_seed(42)
train_dataset, val_dataset = random_split(raw_train_dataset,
[train_size, val_size], generator=generator)
loss_fn = nn.CrossEntropyLoss();print(' ',loss_fn)

batch_size = 128
train_loader = DataLoader(train_dataset, batch_size=batch_size,
shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=10000, shuffle=False)
test_loader = DataLoader(raw_test_dataset, batch_size=10000,
shuffle=False)
def test(model, loss_fn, test_loader, device):
    model.eval() # Turn off layers like dropout or batchnorm for
evaluation
    running_loss = 0.0
    correct_predictions = 0

    with torch.no_grad():
        for i, (inputs, labels) in enumerate(test_loader):
            inputs = inputs.to(device)
            labels = labels.to(device)
            outputs = model(inputs)
            loss = loss_fn(outputs, labels)

            # Multiply by inputs.size(0) to get the total loss of the
current batch
            running_loss += loss.item()

            _, predicted = torch.max(outputs.data, 1)
            correct_predictions += (predicted == labels).sum().item()

        # Divide total accumulated loss by the total number of samples
        avg_test_loss = running_loss / len(test_loader)
        test_accuracy = correct_predictions / len(test_loader.dataset)

    return avg_test_loss, test_accuracy
def
train(model,loss_fn,train_loader,device,optimizer,num_epochs,val_loader)
:
    print('inside the function')
    history = {'loss': [], 'val_loss': [], 'accuracy': [], 'val_accuracy':
[]}]
    for epoch in range(num_epochs):
        if epoch == 0:
            print('in 1st loop')
            model.train()
            running_loss = 0.0 #배치마다 나온 로스값을 누적하기 위한 플레이스 홀
더. avg_train_loss의 발사대
            correct_predictions = 0 #Accuracy 측정을 위해

```

```

        progress_bar = tqdm(
            train_loader, desc=f"Epoch [{epoch+1}/{num_epochs}]",
            leave=True
        )
    for i, (inputs, labels) in enumerate(progress_bar):
        # if i == 0:
        #     print('im in 1st batch')
        optimizer.zero_grad() #이전 배치에서의 경사를 초기화
        inputs = inputs.to(device)
        labels = labels.to(device)
        outputs = model(inputs)
        loss = loss_fn(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
        _, predicted = torch.max(outputs.data, 1)
        correct_predictions += (predicted == labels).sum().item()
        progress_bar.set_postfix({"batch_loss": f"{loss.item():.4f}"})

    avg_train_loss = running_loss / len(train_loader)
    train_accuracy = correct_predictions / len(train_loader.dataset)
    history['loss'].append(avg_train_loss)
    history['accuracy'].append(train_accuracy)
    val_loss, val_accuracy = test(model, loss_fn, val_loader, device)
    history['val_loss'].append(val_loss)
    history['val_accuracy'].append(val_accuracy)
    progress_bar.set_postfix({"batch_loss": f"{loss.item():.4f}"})
    print(
        f"Epoch {epoch+1}/{num_epochs} | "
        f"Train Loss: {avg_train_loss:.4f} | "
        f"Train Acc: {train_accuracy*100:.1f}% | "
        f"Val Loss: {val_loss:.4f} | "
        f"Val Acc: {val_accuracy*100:.1f}%"
    )
    return history

def show_history(history):
    plt.figure(figsize=(10, 5))
    plt.plot(history['loss'], label='Train Loss')
    plt.plot(history['val_loss'], label='Val Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title('Loss Curve (PyTorch)')
    plt.legend()
    plt.grid(True)
    plt.show()

    plt.figure(figsize=(10, 5))
    plt.plot(history['accuracy'], label='Train Accuracy')
    plt.plot(history['val_accuracy'], label='Val Accuracy')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.title('Accuracy Curve (PyTorch)')
    plt.legend()
    plt.grid(True)
    plt.show()

def print_test(model):
    X_test, target = next(iter(test_loader))

```

```

X_test = X_test.to(device)
target = target.to(device)
with torch.no_grad():
    foo = model.forward(X_test)
    _,pred = foo.max(dim=1)
    metric = torchmetrics.ConfusionMatrix(task="multiclass",
num_classes=10).to(device)
    cm = metric(pred, target)
    metric.plot()
    accuracy = torch.trace(cm) / cm.sum()
    plt.title(f'Accuracy: {(100*accuracy.item()):.3f}%')
    model_dict[model.__class__.__name__] = model
    return cm

```

CrossEntropyLoss()

```
In [10]: model = MLP() # @param ["Perceptron()", "MLP()", "MLP_naive()",
"FlexibleMLPSequential([28*28,10])", "HighPerfCNN()", "LowParamCNN()"]
{allow-input: true,type:"raw"}
model.to(device)
optimizer = optim.Adam(model.parameters());

print(summary(model,(1,1,28,28),verbose=0,col_names=["input_size",
"output_size", "num_params", "mult_adds", "trainable"]))

num_epochs = 20
```

```
=====
=====
=====
Layer (type:depth-idx)      Input Shape
Output Shape      Param #      Mult-Adds
Trainable
=====
=====
=====
MLP                      [1, 1, 28, 28]
[1, 10]              --              --
True
└─Linear: 1-1          [1, 784]
[1, 128]             100,480          100,480
True
└─ReLU: 1-2           [1, 128]
[1, 128]             --              --
--
└─Linear: 1-3          [1, 128]
[1, 64]              8,256          8,256
True
└─ReLU: 1-4           [1, 64]
[1, 64]              --              --
--
└─Linear: 1-5          [1, 64]
[1, 10]              650          650
True
=====
=====
=====
Total params: 109,386
Trainable params: 109,386
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 0.11
=====
=====
=====
Input size (MB): 0.00
Forward/backward pass size (MB): 0.00
Params size (MB): 0.44
Estimated Total Size (MB): 0.44
=====
=====
=====
```

```
In [11]: history =  
train(model,loss_fn,train_loader,device,optimizer,10,val_loader)  
show_history(history)  
print_test(model)
```

inside the function
in 1st loop

Epoch [1/10]: 100%|██████████| 391/391 [00:07<00:00, 55.82it/s,
batch_loss=0.1792]

Epoch 1/10 | Train Loss: 0.4547 | Train Acc: 87.9% | Val Loss:
0.2379 | Val Acc: 93.0%

Epoch [2/10]: 100%|██████████| 391/391 [00:08<00:00, 47.47it/s,
batch_loss=0.0664]

Epoch 2/10 | Train Loss: 0.1813 | Train Acc: 94.7% | Val Loss:
0.1660 | Val Acc: 95.2%

Epoch [3/10]: 100%|██████████| 391/391 [00:07<00:00, 49.27it/s,
batch_loss=0.1137]

Epoch 3/10 | Train Loss: 0.1295 | Train Acc: 96.2% | Val Loss:
0.1459 | Val Acc: 95.7%

Epoch [4/10]: 100%|██████████| 391/391 [00:06<00:00, 56.06it/s,
batch_loss=0.0412]

Epoch 4/10 | Train Loss: 0.0998 | Train Acc: 97.0% | Val Loss:
0.1231 | Val Acc: 96.3%

Epoch [5/10]: 100%|██████████| 391/391 [00:07<00:00, 52.92it/s,
batch_loss=0.1528]

Epoch 5/10 | Train Loss: 0.0795 | Train Acc: 97.6% | Val Loss:
0.1111 | Val Acc: 96.7%

Epoch [6/10]: 100%|██████████| 391/391 [00:07<00:00, 50.89it/s,
batch_loss=0.0740]

Epoch 6/10 | Train Loss: 0.0661 | Train Acc: 98.0% | Val Loss:
0.1022 | Val Acc: 96.9%

Epoch [7/10]: 100%|██████████| 391/391 [00:07<00:00, 53.73it/s,
batch_loss=0.0809]

Epoch 7/10 | Train Loss: 0.0533 | Train Acc: 98.3% | Val Loss:

0.0974 | Val Acc: 96.9%

Epoch [8/10]: 100%|██████████| 391/391 [00:07<00:00, 51.02it/s,
batch_loss=0.0674]

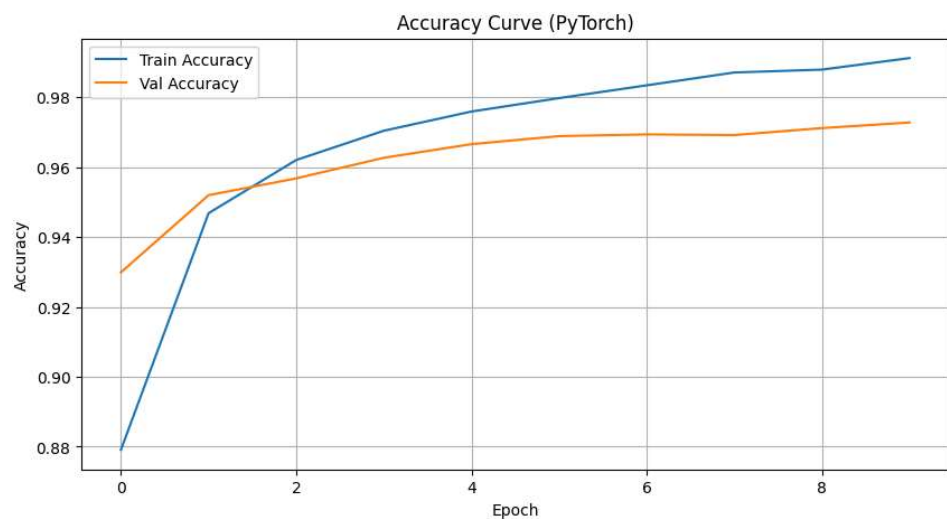
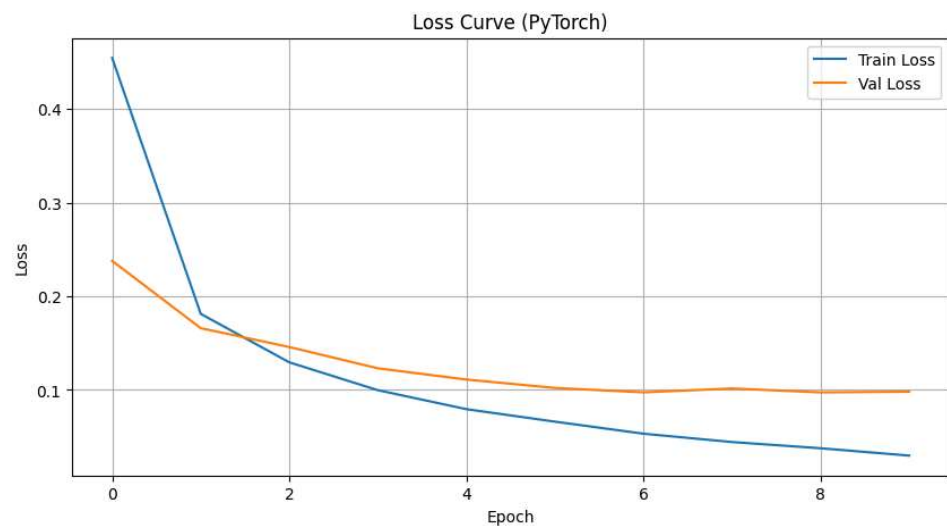
Epoch 8/10 | Train Loss: 0.0444 | Train Acc: 98.7% | Val Loss:
0.1016 | Val Acc: 96.9%

Epoch [9/10]: 100%|██████████| 391/391 [00:07<00:00, 51.00it/s,
batch_loss=0.0097]

Epoch 9/10 | Train Loss: 0.0377 | Train Acc: 98.8% | Val Loss:
0.0974 | Val Acc: 97.1%

Epoch [10/10]: 100%|██████████| 391/391 [00:07<00:00,
53.05it/s, batch_loss=0.0156]

Epoch 10/10 | Train Loss: 0.0299 | Train Acc: 99.1% | Val Loss:
0.0982 | Val Acc: 97.3%

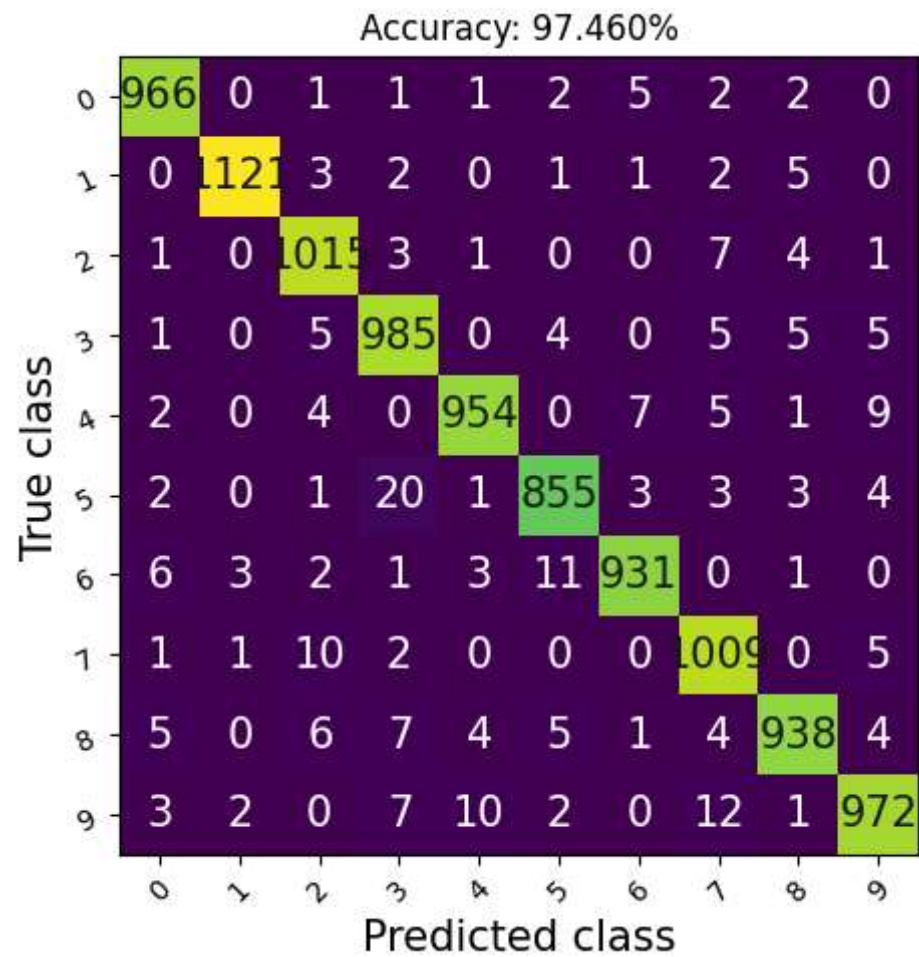


tensor([[966, 0, 1, 1, 1, 2, 5, 2, 2,
0],

```

0],
1],
5],
9],
4],
0],
5],
4],
972]],
device='cuda:0')

```



증강 데이터 테스트

```
In [8]: def html_canvas():
    from IPython.display import HTML, display
    import base64

    from google.colab import output

    # HTML + JavaScript 코드
    canvas_html = """
    <canvas id="canvas" width="280" height="280" style="border:1px solid
black;"></canvas><br>
    <button onclick="saveCanvas()">Save as PNG</button>
    <button onclick="clearCanvas()">Clear Canvas</button>
    <script>
    let canvas = document.getElementById('canvas');
    let ctx = canvas.getContext('2d');
    let drawing = false;

    canvas.addEventListener('mousedown', (e) => { drawing = true;
ctx.beginPath(); });
    canvas.addEventListener('mouseup', () => { drawing = false; });
    canvas.addEventListener('mousemove', (e) => {
        if (!drawing) return;
        ctx.lineWidth = 10;
        ctx.lineCap = 'round';
        ctx.strokeStyle = 'black';
        let rect = canvas.getBoundingClientRect();
        ctx.lineTo(e.clientX - rect.left, e.clientY - rect.top);
        ctx.stroke();
    });

    function saveCanvas() {
        let dataURL = canvas.toDataURL('image/png');
        google.colab.kernel.invokeFunction('notebook.save_canvas',
[dataURL], {});
    }

    function clearCanvas() {
        ctx.clearRect(0, 0, canvas.width, canvas.height);
    }
    </script>
    """

    # 데이터를 저장하는 Python 콜백 함수
    def save_canvas(data):
        png_data = base64.b64decode(data.split(',')[1])
        with open('canvas.png', 'wb') as f:
            f.write(png_data)
        print("Saved as canvas.png")
```

```

# Colab 콜백 함수 등록
output.register_callback('notebook.save_canvas', save_canvas)

# HTML 렌더링
display(HTML(canvas_html))
def get_tensor_from_png():
    ''' colab 구글드라이브에 저장된 png를 텐서로 바꿈
    Args:
    Returns:
        tensor: (1,28,28) size
        ...

    from PIL import Image
    image = Image.open('canvas.png').convert('RGBA')
    image = np.array(image.resize((28, 28))).astype('float32') / 255.0
    image = image[:, :, 3] # 알파 채널
    transform = transforms.ToTensor()
    tensor = transform(image)
    tensor = tensor.reshape(1,28,28)
    return tensor
def get_tensor_agumented_test(ind_train=42,xpxl=0,ypxl=0):
    '''translation을 통해 test 데이터를 살짝 변형시킨 인풋 생성
    Args:
        ind_train (int):
        xpxl (int):
        ypxl (int):
    Returns:
        tensor (tensor): (1,28,28) size
        label (int):
        ...

    tensor,label=raw_test_dataset[ind_train]
    tensor = torch.roll(tensor, shifts=(xpxl, ypxl), dims=(1, 2))
    return tensor, label
def vis_tensor(tensor):
    '''tensor를 plt로 시각화
    Args:
        tensor (tensor): (1,28,28) size
    Returns:
        ...

    arr = tensor.detach().cpu().numpy()
    arr = arr.reshape(28,28)
    plt.imshow(arr)
def get_predict(model,tensor):
    ''' model의 아웃풋 출력
    Args:
        model (model): forward를 가지고 있음
        tensor (tensor): (1,28,28) size
    Returns:
        predict (array): (10,)
        ...

    tensor = tensor.to(device)
    tensor = tensor.view(1,1,28,28)
    with torch.no_grad():
        x_forward = model.forward(tensor)
        predict = torch.softmax(x_forward, dim=-1).detach().cpu().numpy()
        predict = predict.squeeze()
    return predict

```

```

def vis_predict(predict):
    ''' visualize predict
    Args:
        predict (array): (10,)
    Returns:
        ...

    plt.bar(np.arange(10),predict)
    plt.xlabel('labels')
    plt.ylabel('probability')
    plt.xticks(np.arange(10))

def compare_model(model_1,model_2,tensor):
    '''compare two model with tensor
    Args:
        model1 (model): forward를 가지고 있음
        model2 (model): forward를 가지고 있음
        tensor (tensor): (1,28,28) size
    Returns:
        ...

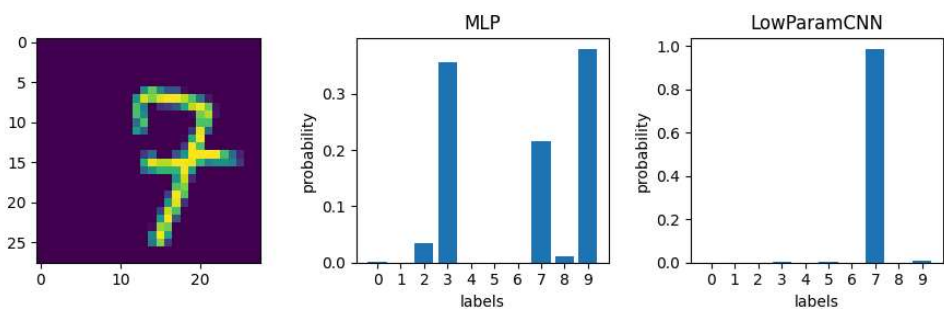
    models = [model_1,model_2]
    fig,axs = plt.subplots(1,3,figsize=(9,3))
    plt.sca(axs[0])
    vis_tensor(tensor)
    for ind in range(2):
        model = models[ind]
        plt.sca(axs[ind+1])
        vis_predict(get_predict(model,tensor))
        plt.title(model.__class__.__name__)
    plt.tight_layout()

```

```

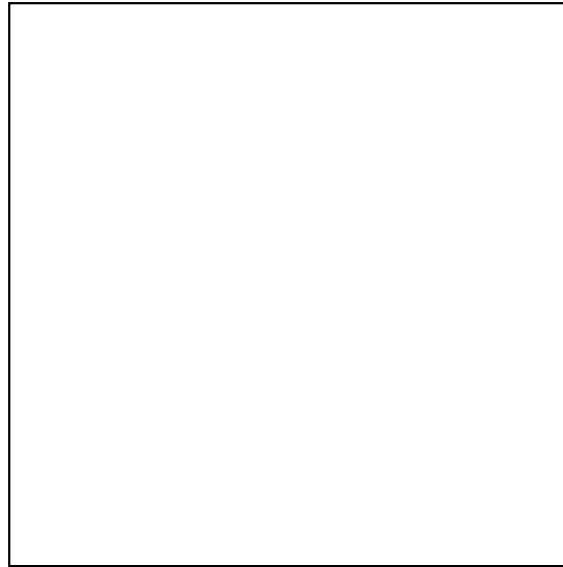
In [13]: ind_train=550 # @param
        xpxl=0# @param
        ypxl=3# @param
        model_1 ="MLP" # @param ["Perceptron", "MLP", "MLP_naive",
        "FlexibleMLPSequential","HighPerfCNN","LowParamCNN"]
        model_2 ="LowParamCNN" # @param ["Perceptron", "MLP", "MLP_naive",
        "FlexibleMLPSequential","HighPerfCNN","LowParamCNN"]
        x,_ = get_tensor_agumented_test(ind_train,xpxl,ypxl)
        compare_model(model_dict[model_1],model_dict[model_2],x)

```



```
In [14]: html_canvas()

# x,label=raw_test_dataset[1]
print(x.shape)
```



Save as PNG

Clear Canvas

torch.Size([1, 28, 28])
Saved as canvas.png

```
In [15]: model_1 ="MLP" # @param ["Perceptron", "MLP", "MLP_naive",
"FlexibleMLPSequential","HighPerfCNN","LowParamCNN"]
model_2 ="LowParamCNN" # @param ["Perceptron", "MLP", "MLP_naive",
"FlexibleMLPSequential","HighPerfCNN","LowParamCNN"]
compare_model(model_dict[model_1],model_dict[model_2],get_tensor_from_png())
```

